SYSTEM ANALYSIS DOCUMENTATION (SAD)

UMHackathon 2026

umhackathon@um.edu.my

Project Name: MyGov-Guard AI

Group Name: Gimme4

Group Member:

1. Gwee Zi Ni
2. Lee Jia Yee
3. Michelle Ho Chia Xin
4. Tay Xin Ying

© UMHackathon 2026

Email: umhackathon@um.edu.my

1. Introduction

This System Analysis Document (SAD) for MyGov-Guard AI defines the technical design and architecture behind the solution described in the Product Requirement Document (PRD).

1.1. Purpose

The purpose of this document is to describe the technical scope and key design decisions behind the development of MyGov-Guard AI.

The key elements of the system that have been covered are as follows:

1. ***Architecture**: The system features a cloud-ready microservices backend combined with a 'Local-First' frontend data architecture. To prioritize user privacy and application performance, we store sensitive scan logs locally on the device rather than a central server, minimizing data breach risks. Key backend services are separated into independent modules, including:*
   - *OCR Service*
   - *Agentic Workflow Service*
   - *Text-to-Speech (TTS) Service*

   Each service operates independently to improve scalability and maintainability.

2. ***Data Flows:** The system processes unstructured inputs (images or text) from the user's mobile device. The data flow follows this pipeline:*

   User Input → Backend API → OCR Processing → Gemini 2.5 Flash Processing → Structured JSON Response → Frontend UI

   Audio Playback Flow: Frontend Request -> Backend TTS Endpoint -> Google Cloud Wavenet Engine -> MP3 Stream -> Expo-AV Player (Frontend)

   This ensures that all outputs are standardized and easily rendered on the user interface.

3. ***Model Process:** The system uses a 3-stage agentic workflow:*
   - *Agent 1: Scam detection (security verification)*
   - *Agent 2: Translation (simplify bureaucratic language)*
   - *Agent 3: Action generation (step-by-step guidance)*

   This sequential design ensures that safety checks are completed before any user instructions are generated.
4. ***Role of Reference:** This document serves as a technical reference for developers and testers. It provides a high-level blueprint for: System architecture, Data flow design, Model integration logic as well asTesting and implementation alignment*

© UMHackathon 2026

1.2. Background

Malaysians frequently receive government-related messages such as LHDN and JPJ and are increasingly exposed to phishing scams. Currently, users rely on manual methods such as hotline verification or physical counter visits, which are slow and inefficient. This system introduces an AI-driven pipeline that processes multimodal inputs in real time to improve verification, understanding, and response.

1.3. Target Stakeholder

Stakeholders	***Roles***	***Expectations***
Citizens / SME Owners	*Upload images or text of government communications to verify authenticity and understand the required actions.*	*A fast, simple experience. Clear distinction between "Scam" (Red) and "Safe" (Green). Easy-to-understand next steps.*
Government Agencies (Indirect)	*Indirect beneficiaries of the system*	*Expect reduced scam victimisation rates and fewer unnecessary counter visits due to better digital understanding among citizens*
Development Team	*Responsible for system and frontend design, backend implementation and deployment*	*Well-organized code (modular microservices), clear communication between the app and server (APIs), and fast, cost-effective AI responses (low latency and controllable LLM costs).*
QA / Testing Team	*Responsible for system validation and edge case testing*	*Clear rules on exactly how the app should behave when things go wrong, especially for edge cases such as blurry images, mixed language and oversized inputs*

This stakeholder mapping ensures alignment between system design, user experience, and operational constraints.

© UMHackathon 2026

2. System Architecture & Design

2.1. High Level Architecture

2.1.1. Overview: MyGov-Guard AI follows a cloud-based microservices architecture

Type	Details
System	*Mobile App React Native*
Architecture	*Hybrid Cloud-Native Architecture* ***Client-Side (React Native Mobile App)*** ● *Local Cache (State Management)* ● *Local On-Device Storage (AsyncStorage)* ***Backend (Cloud Microservices)*** ● *API Gateway / Core Backend (FastAPI)* ● *OCR Service* ● *TTS Voice Synthesis Service (Google Cloud WaveNet)* ● *Agentic Workflow Service*

MyGov-Guard AI is primarily structured as a client-server-based system featuring a Citizen Mobile Application (React Native) that communicates via a Backend API layer (FastAPI). To prioritize user privacy and application performance, we adopted a 'Local-First' data architecture. The application utilizes AsyncStorage as a lightweight and asynchronous key-value storage system. This acts as a decentralized local database, ensuring that user data—such as scan history, personalized names, and language preferences—remains intact even after the application is closed or the device is restarted, ensuring near-instant access.

2.1.2 LLM as Service Layer

The system uses Gemini 2.5 Flash API as the core reasoning engine.
Prompting Strategy: We use a multi-step agentic workflow:

- Step 1: Scam Detection Agent (security check first).
- Step 2: Translation Agent (simplify content).
- Step 3: Action Agent (generate step-by-step instructions)

© UMHackathon 2026

2.1.3 Dependency Diagram

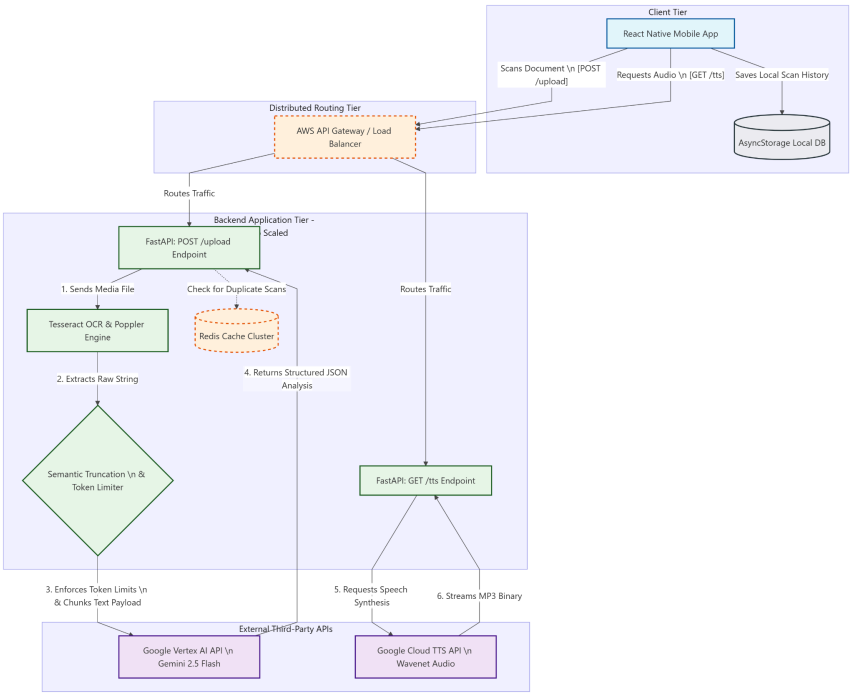

Figure 2.1.3.1: Dependency Diagram for MyGov-Guard AI

2.1.4 Sequence Diagram

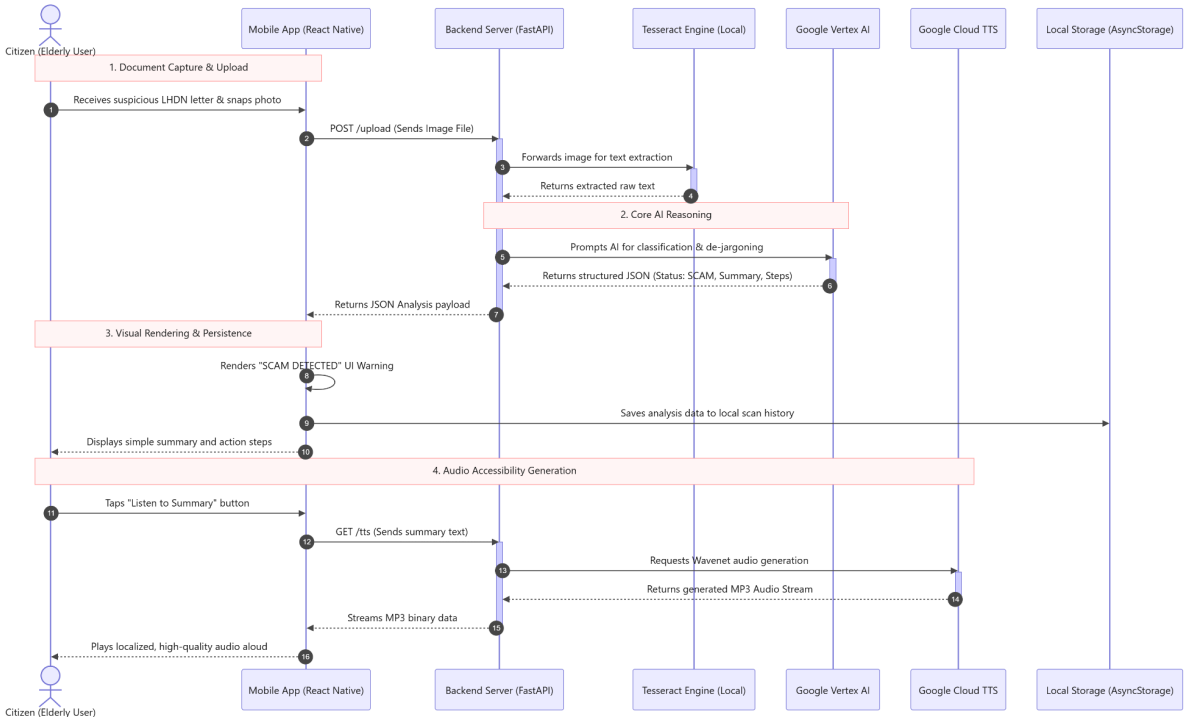

Figure 2.1.4.1: Sequence diagram for MyGov-Guard AI

© UMHackathon 2026

Email: umhackathon@um.edu.my

2.2. Technological Stack

- ***2.2.1. Frontend:*** *React Native (Expo)*
- ***2.2.2. Backend:*** *FastAPI (Python)*
- ***2.2.3. Data Persistence:*** *AsyncStorage (Decentralized Local Key-Value Store)*
- ***2.2.4. Storage:*** *Local Device Temporary Storage (Images are auto-deleted post-processing for PDPA standards)*
- ***2.2.5. OCR:*** *Tesseract*
- ***2.2.6. LLM:*** *Gemini 2.5 Flash API*
- ***2.2.7. Hosting:*** *Local Sandbox*
- ***2.2.8. TTS Engine:*** *Google Cloud Text-to-Speech (Premium Wavenet Voices)*

2.3. Key Data Flows

2.3.1. Overall Data Flow

Our system's dataflow begins when you upload an image or PDF via the Expo frontend. This file is transmitted to the FastAPI backend, where the Tesseract OCR engine (and Poppler for PDFs) converts the visual content into raw, machine-readable text.

Next, this text is passed to Google Vertex AI (Gemini). The AI analyzes the content against Malaysian-specific scam detection rules—such as looking for unofficial domains or bank details—and classifies it as safe, a warning, or a scam. It then generates a structured JSON response containing a summary and protective "next steps," strictly localized into your selected language (English, Malay, or Chinese).

Finally, when you interact with the result, the frontend uses expo-av to handle audio. If you tap the "Play" button, the app sends a request to a dedicated /tts endpoint on the backend. This endpoint triggers Google Cloud Text-to-Speech to generate a high-quality MP3 stream using premium Wavenet voices, which is then streamed back to your phone for real-time playback and progress tracking.

© UMHackathon 2026

2.3.1.1. Data Flow Diagram (DFD)

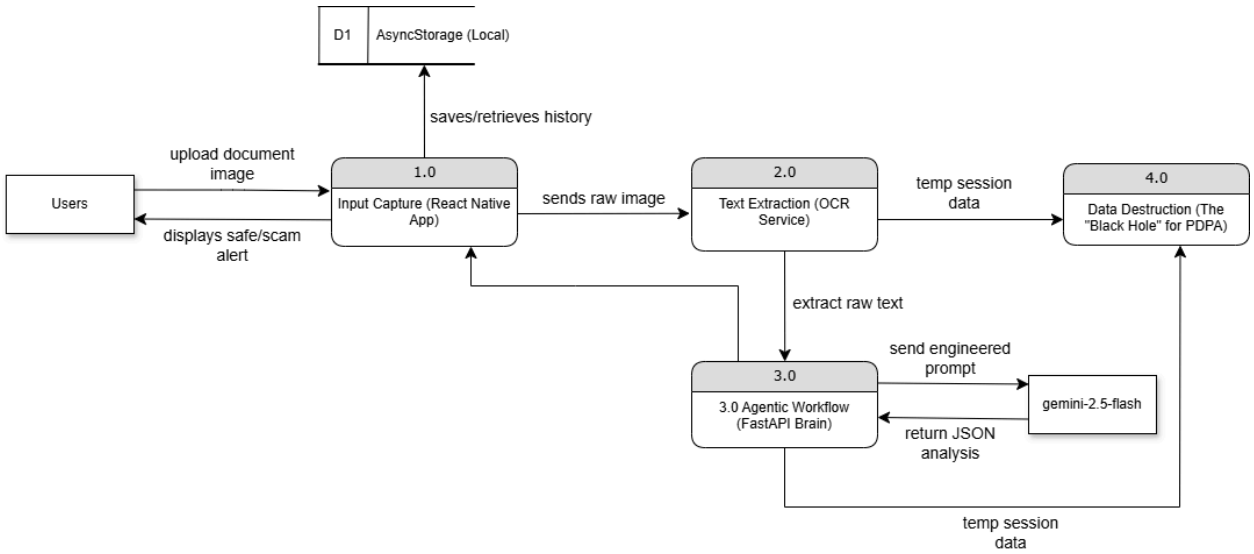

Figure 2.3.1.1.1: Data Flow Diagram (DFD) for MyGov-Guard AI

2.3.2. Normalized Database Schema

Because the system utilizes a Local-First data architecture via AsyncStorage rather than a centralized relational database, a 3NF Entity-Relationship Diagram (ERD) is not applicable. To ensure maximum data protection, data is persisted locally on the user's device in lightweight JSON key-value structures.

For example, the primary user_history key stores an array of objects structured as follows:

- id: Unique scan identifier (Unix timestamp string)
- title: Original filename of the scanned document
- date: Date and time of the document scan (ISO 8601 string format)
- status: Classification result ("safe", "scam", "warning", or "error")
- summary: The localized, plain-language analysis generated by the Gemini 2.5 Flash API
- steps: Array of strings detailing the recommended SOP/next steps

© UMHackathon 2026

3. Functional Requirements & Scope

This section highlights the core boundaries of the projects, focusing on a high-impact MVP to showcase the technical feasibility of the project.

3.1. Minimum Viable Product

3.1.1. *The following 4 core features will be developed and demonstrated by the end of UMHackathon 2026.*

#	Features	Description
1	Scam Detection	The Gemini 2.5 Flash AI acts as a security auditor to analyze the document's text and linguistic patterns. It utilizes a dynamic risk-scoring algorithm that evaluates factors such as demands for personal bank transfers, manufactured urgency, requests for sensitive information, and the presence of official government domains (e.g., .gov.my) to classify the input as Potential Risk (orange), Scan Failed (grey), Official Verified (green) or Scam Detected (red).
2	Translation Engine	The AI engine translates formal, complex language or jargon into simplified "plain language" summaries. It also localizes the output into the user's preferred language (English, Malay, or Chinese) to ensure the user fully understands the document's context and intent.
3	Action Guide Generator	Based on the interpreted content, the system generates a localized, step-by-step SOP (Standard Operating Procedure). It guides the user through the recommended next steps to safely resolve the matter or verify the document's authenticity.
4	Multi-modal input	Users can upload screenshots of digital messages, photos of physical letters, or PDF documents. The system utilizes Tesseract OCR and Poppler to accurately extract text from the files before processing it with Gemini AI. Additionally, the system features a Text-to-Speech (TTS) engine powered by Google Cloud to audibly read the analysis back to the user in their preferred language.

© UMHackathon 2026

3.2. Non-Functional Requirements (NFRs)

Here, highlights the system qualities that are not features but are crucial for reliability and improve the system. [e.g. table in below]

Quality (e.g.)	***Requirements (e.g.)***	***Implementation (e.g)***
Scalability	*The system must be able to handle peak traffic, such as during national tax deadlines or new government aid (e.g., PADU) rollouts.*	*The FastAPI backend is specifically engineered to be stateless. For production deployment, this stateless architecture is designed to be hosted on Google Cloud Run and placed behind Google Cloud Load Balancing to dynamically distribute traffic based on CPU/Memory load.*
Privacy	*Personally Identifiable Information (PII) such as IC numbers or home addresses must NOT be stored or used for model training to ensure citizen data protection.*	*The system leverages Google Cloud Vertex AI for the backend, which strictly enforces enterprise data privacy and does not use customer prompts for model training. Furthermore, the FastAPI backend operates completely stateless, meaning session data and uploaded documents are discarded immediately after the analysis is returned to the user.*
Latency	*The end-to-end process should return a final action guide within 4-6 seconds for text, image, and PDF document uploads.*	*Utilize Asynchronous API calls (async/await) for the Gemini 2.5 Flash processing. Display a user-facing loading state and progress indicators on the frontend while awaiting the multi-step reasoning results.*
Cost	*The operational cost of LLM tokens per user session must be minimized to ensure the project's long-term sustainability.*	*The system utilizes the gemini-2.5-flash model, which is highly optimized for fast, cost-efficient reasoning. A Redis caching layer is planned for future implementation to bypass full-text reasoning for standardized, frequently seen government letters.*

© UMHackathon 2026

3.3. Out of Scope / Future Enhancements

This section identifies the boundaries of the current MVP and the high-level features planned for the "Grand Vision" of MyGov-Guard AI that are not part of the UMHackathon 2026 delivery.

Out of Scope

The following items are intentionally excluded to focus on the core AI reasoning and security features:

- ***In-App Payment System**: To ensure maximum user security and avoid financial liability, the system will redirect users to verified official payment portals rather than processing transactions within the app.*
- ***Direct Government Database Access**: The current prototype will rely on a simulated/curated SOP database and publicly available data rather than private, live API integrations with internal government systems (e.g., LHDN or JPJ internal records).*
- ***Human Legal Consultation**: The system provides AI-driven explanations and guidance based on public documentation; it does not facilitate real-time chat or consultations with human lawyers or legal experts.*

© UMHackathon 2026

4. Monitor, Evaluation, Assumptions & Dependencies

4.1. Technical Evaluation (Post-Deployment Technical Evaluation Strategy)

While the current preliminary delivery is a localized Minimum Viable Product (MVP), the following strategies outline our technical roadmap for safely deploying MyGov-Guard AI to the public in a production environment.

4.1.1. Grayscale Rollout & A/B Testing

To ensure the stability and accuracy of the multi-agent workflow, MyGov-Guard AI will employ a highly controlled deployment strategy to minimize risk to end-users:

- ***Grayscale Rollout:*** *New critical updates, such as adjustments to the Gemini risk-scoring prompt or OCR extraction parameters, will initially be released to only 5% of the active user base. During this phase, quality assurance protocols will strictly monitor system logs for AI hallucinations, OCR failure rates, and API latency. If the error rate remains below the acceptable 1% threshold, traffic will be gradually increased to 20%, 50%, and finally 100%.*
- ***A/B Testing:*** *Once the baseline system is fully deployed, A/B testing will be conducted to optimize the user interface for our target demographic (e.g., the elderly). For instance, Variant A will present the AI Action Guide output as standard bulleted text, while Variant B will display it as large-font, styled UI step boxes. We will track user engagement and completion rates to determine which presentation strategy yields better comprehension.*

4.1.2. Emergency Rollback (ER) & Golden Release

Given the sensitive nature of government communications and scam detection, system reliability is non-negotiable.

- ***Emergency Rollback (ER):*** *If something goes seriously wrong in production—like the AI service slowing down too much or the scam detection system suddenly missing threats—the system will automatically step in. It quickly rolls everything back to a previous stable version through our CI/CD pipeline, minimizing disruption and risk.*
- ***Golden Release & Fallbacks:*** *The traffic management system will instantly reroute users to the "Golden Release" (the last known, highly tested, and completely stable version of the application). Additionally, the system is hardcoded with a Multi-language Safety Net. If the upstream Google Vertex AI services suffer a total outage or network timeout, the system automatically catches the exception and displays a fail-safe UI. This static fallback temporarily guides the user with basic safety steps and localized error messages without crashing the app.*

4.1.3. Priority Matrix

4.1.3.1. *To ensure systematic handling of system anomalies and maintain user trust, the MyGov-Guard AI system utilizes the following Priority Matrix to dictate trigger conditions and automated/manual actions.*

© UMHackathon 2026

Priority Level	*Definition*	*Trigger Conditions*	*Automated / Manual Actions*
P1 (Critical)	*Core system failure or security breach impacting all users.*	*• Google Vertex AI (Gemini) downtime exceeds 2 minutes.* *• False Negative rate for Scam Detection spikes above 2%.*	***Action:*** *Automated Emergency Rollback (ER) to Golden Release.* ***Fallback (Currently Implemented):*** *The system automatically catches API exceptions to prevent app crashes. It immediately displays a localized, static fail-safe UI that warns the user and directs them to call the official NSRC 997 hotline.* ***Production Phase Goals:*** *Auto-disable AI upload features instantly via remote feature flags and trigger automated alerts to page on-call engineers.*
P2 (High)	*Major feature degradation, but the system is partially usable.*	*• OCR extraction confidence drops below 50% for 5 minutes.* *• Text-to-Speech (TTS) Cloud API latency exceeds 8 seconds.*	***Action (Currently Implemented):*** *The system isolates feature-specific errors without crashing the main application. If OCR fails, a localized "Scan Failed" UI prompts the user to upload a clearer image. If the TTS API times out, an*

© UMHackathon 2026

			"Audio Error" alert is shown, but the AI text analysis remains fully functional and readable. ***Production Phase Goals:*** *Automatically disable degraded UI modules (e.g., temporarily hiding the "Listen" button) and trigger AWS CloudWatch alerts to notify the backend engineering team.*
P3 (Medium)	*Non-critical component failure or performance bottleneck.*	*• Planned Template Caching Service (Redis) goes offline.* *• Minor API rate-limiting reached on secondary Google Cloud services.*	***Action (Currently Implemented):*** *The FastAPI server heavily leverages asynchronous endpoints (async def) to ensure that API bottlenecks or network delays do not block the main event loop, allowing the server to continue handling other users smoothly.* ***Production Phase Goals:*** *Traffic automatically bypasses the offline cache layer and routes directly to the primary Gemini API. AWS Auto Scaling automatically spins up additional*

© UMHackathon 2026

			backend instances to handle the load.
P4 (Low)	*Minor UI/UX bugs or non-breaking formatting issues.*	*• Gemini AI JSON output has minor formatting anomalies (e.g., missing bullet points in the Action Guide).* *• Slow loading of UI assets.*	***Action (Currently Implemented):*** *The React Native frontend is designed to safely ignore minor JSON formatting errors, ensuring the user can still read the core scam analysis summary without experiencing a blank screen.* ***Production Phase Goals:*** *No emergency action required. Errors and slow asset loads are silently logged to an automated bug tracker (e.g., Sentry / Jira) for the QA team to review and patch in the next standard development sprint.*

© UMHackathon 2026

4.2. Monitoring

To guarantee the high availability and accuracy of the MyGov-Guard AI system, a comprehensive, multi-layered monitoring plan is implemented. The system utilizes automated tools to continuously track infrastructure health, Google Cloud Vertex AI integrations, and the quality of the Tesseract OCR engine.

*If any anomaly surpasses our predefined thresholds, automated alerts are instantly routed to the engineering team for mitigation. We segment our monitoring into **four critical layers**:*

Monitoring Layer	*Component / Tool*	*Metric Monitored*	*Trigger Threshold*	*Automated Action & Alert*
1. External AI Services (P1)	***Google Vertex AI (Gemini)***	*API Downtime & Scam Detection Accuracy*	*Gemini API downtime > 2 mins OR False Negative rate > 2%*	***Action:*** *Automated Emergency Rollback. Disable AI scan features instantly. Display static UI redirecting users to the official NSRC 997 hotline.*
2. Audio & TTS Service (P2)	***Google Cloud TTS (Wavenet)***	*Audio Generation Latency*	*TTS latency exceeds 8 seconds*	***Action:*** *Auto-disable the "Listen" button in the UI. Force users to rely on text-only summary. Trigger CloudWatch alerts to the backend team.*
3. OCR & App Logic (P2)	*FastAPI & Tesseract OCR*	*OCR Extraction Confidence*	*Confidence drops below 50% for 5 mins*	***Action:*** *Auto-hide the "Upload Image/PDF" cards. Display "Manual Input Only" mode. Page on-call engineers for*

© UMHackathon 2026

				Tesseract threshold review.
4. Infrastructure Health (P3)	*Local Compute / API Server*	*CPU & Rate Limiting*	*CPU > 80% OR Secondary API rate-limiting reached*	***Action:*** *Bypass non-essential cache layers. Trigger scaling logic (AWS Auto Scaling) to spin up additional instances. Log for next-day review.*
5. AI Business Logic (P4)	***Summary Formatting***	*JSON Output Integrity*	*Missing bullet points in "Next Steps" or minor formatting bugs*	***Action:*** *No emergency action. Errors are silently logged to the bug tracker (Sentry) for the QA team to patch in the next standard sprint.*

4.2.1. Alerting Workflow

When system damage or degradation is detected, the workflow operates automatically to ensure minimal downtime:

1. ***Detection:*** *Google Cloud Monitoring or Sentry detects a metric breaching the trigger threshold (e.g., Gemini API timeout or Cloud Text-to-Speech latency).*
2. ***Evaluation:*** *The monitoring agent checks the Priority Matrix to determine severity (P1 to P4).*
3. ***Execution:*** *For critical errors, the system triggers automated actions such as disabling the audio player in the Expo UI or initiating an emergency container restart via GCP Cloud Run.*
4. ***Notification:*** *PagerDuty instantly sends an SMS/Email to the on-call engineers for manual intervention.*

© UMHackathon 2026

4.3. *Assumptions*

To clearly define the operational boundaries and scope of the MyGov-Guard AI system, the following key assumptions were made during the development and deployment phases:

1. ***User & Hardware Assumptions:***
   - ***Stable Connectivity:*** *Users have a reliable 4G/5G or Wi-Fi connection. This is critical because the app must transmit image data to the FastAPI backend and stream high-quality audio files from the GCP TTS service.*
   - ***Camera Quality:*** *The user's mobile device has a functioning camera capable of capturing clear, high-resolution photos. The OCR engine performs best when text is sharp and unblurred.*
   - ***Document Integrity:*** *It is assumed that physical letters or digital screenshots are legible and well-lit. While our Tesseract OCR logic is robust, it assumes documents are not physically illegible (e.g., severely crumpled, water-damaged, or extremely low-contrast), as a baseline level of text is required for Gemini AI to perform an accurate scam analysis.*
   - ***Audio Output:*** *The user has a functioning speaker or headphones and has enabled "Local Network" permissions on iOS to allow the Expo-AV player to communicate with the local backend server.*

2. ***Technical & Infrastructure Assumptions:***
   - ***API Availability:*** *External core dependencies—specifically Google Vertex AI (Gemini) and Google Cloud Text-to-Speech—maintain an uptime of 99.9% and process requests within acceptable latency limits (< 8 seconds).*
   - ***OCR Accuracy:*** *It is assumed that uploaded images have sufficient resolution (at least 300 DPI) for the Tesseract OCR engine to extract text with a confidence score of at least 60%.*
   - ***Consistency of Official Data:*** *Malaysian government domains (e.g., *.gov.my) and official hotline numbers (e.g., NSRC 997) remain consistent and are not subjected to sudden, unannounced changes that would invalidate the AI Action Guide.*
3. ***Operational & Legal Assumptions:***
   - ***User Consent:*** *Users explicitly grant the application permission to access their Camera and Local Storage. They consent to the temporary processing of their documents on the condition that PII (Personally Identifiable Information) is processed in-memory and not permanently stored.*
   - ***Language & Voice Limitations:*** *While the Cloud TTS module supports high-quality Mandarin (cmn-CN), Malay (ms-MY), and English (en-US), it assumes the user understands these standard accents.*

© UMHackathon 2026

- ***Connectivity:*** *The system assumes a stable internet connection for reaching the FastAPI backend and Google Cloud endpoints; however, the UI will provide localized error messages if the connection fails.*

4.4. External Dependencies

The MyGov-Guard AI system relies on several external services and APIs to process multimodal inputs and execute its agentic workflow. The following table outlines these dependencies, their purposes, and the mitigation strategies for associated risks.

Tools	*Purpose*	*Risks & Mitigation Strategies*
Google Vertex AI (Gemini)	***Core LLM Reasoning Engine:*** *Executes prompt inference, performs multi-language scam classification, simplifies jargon, and generates structured JSON output.*	***High Risk:*** *API rate limits or downtime will disable the analysis feature.* ***Mitigation:*** *Implement a "Static Safe Mode." If Gemini fails, the UI instantly disables uploads and redirects users to the official NSRC 997 hotline.*
Tesseract OCR & Poppler	***Document Processing:*** *Extracts text from user-uploaded images and PDFs (e.g., LHDN, JPJ notices) using the locally hosted Tesseract engine.*	***Medium Risk:*** *Low confidence extraction due to blurry or poorly lit photos.* ***Mitigation:*** *If no readable text is found, the system returns a localized "Incomplete Data" state, prompting the user to take a clearer photo or try a shorter screenshot.*
Google Cloud Text-to-Speech	***Premium Audio Engine:*** *Generates localized, high-quality* ***Wavenet*** *MP3 files for the "Listen" feature (English, Malay, Chinese).*	***Medium Risk:*** *Latency or API connectivity issues on the /tts endpoint.* ***Mitigation:*** *Implement an 8-second timeout. If generation fails, the UI hides the "Listen" button and*

© UMHackathon 2026

		notifies the user that audio is currently unavailable while keeping the text summary visible.
Google Cloud (GCE/Cloud Run)	*Hosting & Compute: Hosts the FastAPI backend logic and manages the local file-to-OCR workflow.*	***Low Risk:*** *Server crashes or unexpected traffic spikes.* ***Mitigation:*** *Use Google Cloud Autoscaling to spin up additional container instances. Images are processed in-memory and not stored permanently to ensure* ***PDPA compliance.***

© UMHackathon 2026

5. Project Management & Team Contributions

The duration of the Preliminary round is approximately 10 days considering the two-week sprint.

5.1. Project Timeline

The sprint is divided into specific development phases to ensure all architectural and testing milestones are met:

Days 1-2 (Ideation & Product Strategy): *Finalizing the problem statement, identifying the target demographic, and drafting the Product Requirements Document (PRD).*

Days 3-4 (System Architecture & Research): *Evaluating technical frameworks (Expo React Native, FastAPI) and finalizing the System Architecture Document (SAD) to ensure cross-platform compatibility and a stateless backend design.*

Days 5-6 (Parallel Development & Mocking): *Work in technical development. Built the React Native UI components and the FastAPI backend routing. Created fake AI responses (mock data) so our frontend team could continue building and testing the app's user interface while we waited for the official API keys from the organizers.*

Day 7 (Changing API & AI Integration): *Noticed the initial AI service was experiencing high latency and frequent connection timeouts. To guarantee a smooth user experience, we made a team decision (after being given permission by the organizers) to switch to Google's Gemini AI for faster and more stable processing.*

Day 8 (Quality Assurance & Pitch Prep): *Connecting the frontend to the live APIs and executing the Quality Assurance Testing Documentation (QATD) protocols. Focused heavily on hallucination handling, OCR edge cases, and mobile UI responsiveness. Filmed and edited the technical demonstration video.*

Day 9 (Deployment Freeze & Finalization): *Code freeze. Finalizing all technical documentation, polishing the pitch deck, and conducting final end-to-end dry runs.*

5.2. Team Members Role

Gwee Zi Ni (Project Lead & Frontend Engineer): *Responsible for the overarching project timeline and developing the cross-platform mobile application using React Native. Focuses on rendering the interactive UI and handling state management on the client side.*

Lee Jia Yee (AI Integration & Backend Engineer): *Focuses on the core logic within the FastAPI backend and the integration of the Google Vertex AI (Gemini) API. Responsible for prompt construction, semantic truncation logic, and parsing the JSON outputs for the frontend.*

© UMHackathon 2026

Tay Xin Ying (UI/UX Designer & Product Strategist): *Responsible for user flow mapping and designing an accessible interface specifically tailored for the target demographic (e.g., elderly users). Manages the A/B testing strategy and user story development.*

Michelle Ho Chia Xin (DevOps & Quality Assurance Lead)*: Manages the local deployment environments, API routing (via Uvicorn), and the secure integration of Google Cloud Platform (GCP) credentials. Responsible for executing the rigorous pass/fail test cases defined in the QATD, focusing heavily on Tesseract OCR extraction accuracy, error handling fallbacks, and tracking AI hallucinations.*

5.3. Recommendations

To ensure MyGov-Guard AI is ready for real-world use by millions of Malaysians, we recommend the following future improvements:

- *Scalability:*

  To support high traffic during viral scam outbreaks, Redis caching can be implemented. When multiple users upload the same message or document, the system can store the AI analysis using a hash of the OCR text. Future requests can then return the cached result instantly without calling the AI again, reducing API cost and improving scalability.

- *Performance:*

  The OCR process can be moved from the backend to the user's device using on-device technologies such as Google ML Kit or iOS Vision API. This reduces the need to upload large image files, as only extracted text is sent to the server. As a result, response time will be faster, especially for users with slower internet connections.

- *Reliability:*

  To ensure continuous system availability, multi-region LLM failover can be implemented using platforms such as Google Vertex AI. If one server region experiences downtime, the system can automatically switch to another region, ensuring uninterrupted scam detection services.

- *System Architecture:*

  Currently, system alerts are hardcoded. In a production environment, these alerts can be stored in a dynamic database such as PostgreSQL or Firebase. A simple content management system (CMS) can be used to update scam alerts in real time without requiring users to update the mobile application.

© UMHackathon 2026

- *User Engagement:*

  Push notification services such as Firebase Cloud Messaging (FCM) and Apple Push Notification Service (APNs) can be integrated. This allows users to receive real-time scam alerts even when the app is not active, improving awareness and response speed.

- *Real-Time Communication:*

  Instead of using a traditional request-response model, technologies like WebSockets or Server-Sent Events (SSE) can be used to push updates directly to users. This reduces unnecessary API calls and ensures that users receive alerts instantly without refreshing the application.

- *User Experience*

  Users can be allowed to subscribe to specific scam categories, such as phishing or banking scams, so they only receive relevant notifications. In addition, a read-tracking feature can be implemented to store alert history, allowing users to review past notifications easily.

- *Automation:*

  The system can be further enhanced by integrating external threat intelligence sources such as Malaysian CERT or PDRM reports. This allows automatic detection of new scam patterns and reduces reliance on manual updates, making the system more efficient and responsive.

© UMHackathon 2026